

MEKELLE UNIVERSITY

ETHIOPIAN INSTITUTE OF TECHNOLOGY MEKELLE (EiT-M)

SCHOOL OF COMPUTING, SOFTWARE ENGINEERING

CLOUD COMPUTING GROUP ASSIGNMENT

3rd Year 2nd Semester

REAL-TIME CLASSROOM MONITORING PLATFORM

Complete Project Documentation

No. Section	Full Name	ID
1 2	Zewde Bekele	UGR/188847/6
2 1	Aklilu Tesfay	UGR/187845/16
3 1	Rediet Tsegay	UGR/188595/16
4 1	Sbhat G/Egzabher	UGR/188642/16
5 1	Kalid Jibril	UGR/188299/16
6 1	Mulugeta Berhe	UGR/188509/16
7 1	Firaol Idris	UGR/188127/16
8 2	Hagazi Berhe	UGR/189722/16

Submission Date: May 2026

"This project is submitted as a partial fulfillment of the Advanced Programming course"

Contents

1 Introduction **8**
- 1.1 Project Overview . . . 8
- 1.2 What Does the Project Do? . . . 8
- 1.3 Problem Statement . . . 8
- 1.4 Project Objectives . . . 8
- 1.5 Target Users . . . 9
- 1.6 Group Members Contribution . . . 9

2 System Architecture **10**
- 2.1 Client-Server Architecture . . . 10
- 2.2 System Components . . . 10
- 2.3 Technology Stack . . . 10

3 Key Terminology **11**
- 3.1 Important Technical Terms . . . 11
- 3.2 Message Protocol Terms . . . 12

4 Features and Functionality **13**
- 4.1 Teacher Dashboard Features . . . 13
  - 4.1.1 Real-time Student Monitoring . . . 13
  - 4.1.2 Attendance Tracking . . . 13
  - 4.1.3 Quiz Management System . . . 13
  - 4.1.4 Announcement System . . . 13
  - 4.1.5 Classroom Control Features . . . 14
  - 4.1.6 Report Generation . . . 14
- 4.2 Student Portal Features . . . 14
  - 4.2.1 Authentication System . . . 14
  - 4.2.2 Quiz Taking Interface . . . 14
  - 4.2.3 Real-time Score Display . . . 14
  - 4.2.4 Announcement Reception . . . 15
  - 4.2.5 Screen Lock/Unlock . . . 15
  - 4.2.6 User Preferences . . . 15

5 How the System Works **16**
- 5.1 System Workflow Diagram . . . 16
- 5.2 Step-by-Step Process Explanation . . . 16
  - 5.2.1 Step 1: Server Startup . . . 16
  - 5.2.2 Step 2: Student Connection . . . 17
  - 5.2.3 Step 3: Authentication . . . 17
  - 5.2.4 Step 4: Heartbeat Mechanism . . . 17
  - 5.2.5 Step 5: Quiz Creation and Sending . . . 18
  - 5.2.6 Step 6: Student Quiz Taking . . . 18
  - 5.2.7 Step 7: Screen Lock During Exam . . . 19

6 Database Design **20**
6.1 Entity Relationship Diagram . . . 20
6.2 Table Structures . . . 20
6.2.1 students Table . . . 20
6.2.2 attendance_log Table . . . 20
6.2.3 quiz_questions Table . . . 21
6.2.4 quiz_responses Table . . . 21
6.2.5 announcements Table . . . 21

7 Installation Guide **22**
7.1 System Requirements . . . 22
7.2 Step-by-Step Installation . . . 22
7.2.1 Step 1: Install Java (Ubuntu) . . . 22
7.2.2 Step 2: Install PostgreSQL . . . 22
7.2.3 Step 3: Create Database . . . 22
7.2.4 Step 4: Download PostgreSQL JDBC Driver . . . 22
7.2.5 Step 5: Install NetBeans IDE . . . 22
7.2.6 Step 6: Create Project in NetBeans . . . 23
7.2.7 Step 7: Add JDBC Driver . . . 23
7.2.8 Step 8: Add Source Code . . . 23
7.2.9 Step 9: Update Database Password . . . 23

8 How to Run the Project **24**
8.1 Running the Teacher Server . . . 24
8.1.1 From NetBeans: . . . 24
8.1.2 From Terminal: . . . 24
8.2 Running the Student Client . . . 24
8.2.1 From NetBeans: . . . 24
8.2.2 From Terminal: . . . 24
8.3 First Time Login . . . 24
8.4 Test Account . . . 25
8.5 Running Multiple Students . . . 25

9 Important Code Explanations **26**
9.1 Multithreading - Handling Multiple Students . . . 26
9.2 Socket Communication . . . 26
9.3 Heartbeat Mechanism . . . 26
9.4 Screen Lock Mechanism . . . 27
9.5 Password Hashing (SHA-256) . . . 27
9.6 Quiz Message Parsing . . . 28
9.7 JavaFX UI Components Used . . . 28

10 Troubleshooting **29**
10.1 Common Errors and Solutions . . . 29
10.2 Debugging Tips . . . 29
10.3 Port Configuration . . . 29
10.4 Firewall Configuration . . . 30

11 Conclusion **31**
11.1 Project Summary 31
11.2 What Has Been Achieved 31
11.3 Skills Demonstrated 31
11.4 Difficulties Encountered and Solutions 31
11.5 Future Enhancements 31

A Appendix A: Complete Database Schema **33**

B Appendix B: Sample Output **35**
B.1 Teacher Dashboard Console Output 35
B.2 Student Portal Output 35

C Appendix C: Project File Structure **36**

References **37**

List of Figures

1 Client-Server Architecture Diagram 10
2 System Workflow Diagram 16
3 Entity Relationship Diagram 20

List of Tables

1 Target Users and Their Roles 9
2 Group Member Contributions 9
3 System Components Description 10
4 Technology Stack 11
6 Network Message Types and Purposes 12
7 students Table Structure 20
8 attendance_log Table Structure 20
9 quiz_questions Table Structure 21
10 quiz_responses Table Structure 21
11 announcements Table Structure 21
12 System Requirements 22
13 Test Account Credentials 25
15 Feature Completion Status 31
16 Difficulties and Solutions 32

1 Introduction

1.1 Project Overview

The **Real-time Classroom Monitoring Platform** is a comprehensive desktop application designed to revolutionize how instructors manage and monitor classroom activities in computer-based learning environments.

1.2 What Does the Project Do?

This system provides a centralized dashboard for teachers to:

- Track student attendance in real-time
- Create and distribute quizzes instantly
- Broadcast announcements to all students
- Lock student screens during examinations
- Monitor which students are active or inactive
- Generate detailed classroom reports

1.3 Problem Statement

Traditional classroom management faces several critical challenges:

1. **Manual Attendance:** Teachers waste 5-10 minutes calling names
2. **No Visibility:** Cannot see which students are actually working
3. **Quiz Distribution:** Paper quizzes are slow and hard to grade
4. **Communication Gap:** No instant way to reach all students
5. **Exam Security:** No way to prevent cheating during online tests

1.4 Project Objectives

The primary objectives of this project are:

1. Create a real-time bidirectional communication system
2. Implement automatic attendance tracking
3. Build an interactive quiz system with instant grading
4. Provide classroom control features (screen lock/unlock)
5. Generate comprehensive performance reports
6. Ensure secure authentication for all users

User Type	Role and Responsibilities
Teacher/Instructor	Creates quizzes, sends announcements, monitors student activity, locks screens, generates reports
Student	Takes quizzes, receives announcements, communicates with teacher, views scores
Administrator	Views reports, monitors overall classroom activity (read-only access)

Table 1: Target Users and Their Roles

1.5 Target Users

1.6 Group Members Contribution

Name	Role/Contribution
Zewde Bekele	Project Coordination, System Architecture Design
Aklilu Tesfay	Lead Developer, Server Side Implementation
Rediet Tsegay	Client Side Development, UI/UX Design
Sbhat G/Egzabher	Database Design and Implementation
Kalid Jibril	Networking and Socket Programming
Mulugeta Berhe	Testing and Quality Assurance
Firaol Idris	Documentation and Report Writing
Hagazi Berhe	Deployment and User Manual

Table 2: Group Member Contributions

2 System Architecture

2.1 Client-Server Architecture

The system follows a **Client-Server Architecture** where the teacher's computer acts as the server and each student's computer acts as a client.

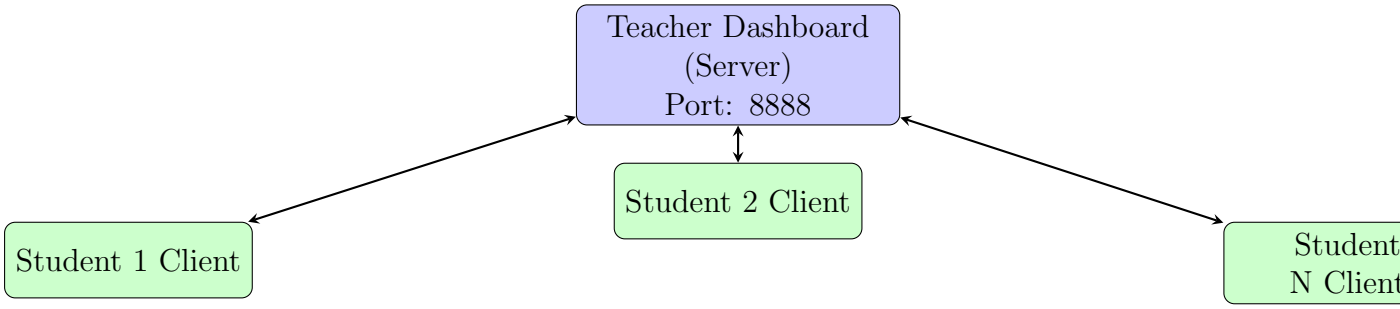


Figure 1: Client-Server Architecture Diagram

2.2 System Components

Component	Description
Teacher Dashboard (Server)	Central control hub that listens for student connections on port 8888, manages multiple student threads, and provides all teacher controls
Student Client	Application running on each student's computer that connects to the teacher, receives quizzes, submits answers, and displays announcements
PostgreSQL Database	Stores all persistent data including student information, attendance logs, quiz questions, and student responses
JavaFX UI	Provides the graphical user interface for both teacher and student applications

Table 3: System Components Description

2.3 Technology Stack

Component	Technology	Purpose
Frontend GUI	JavaFX	Cross-platform graphical user interface
Backend Logic	Java (JDK 17+)	Core business logic and networking
Network Communication	Java Sockets (TCP/IP)	Real-time data transfer between teacher and students
Concurrency	Java Multithreading	Handling multiple student connections simultaneously
Database	PostgreSQL 12+	Persistent data storage
Database Connectivity	JDBC	Java to PostgreSQL connection
Development IDE	Apache NetBeans	Code development and debugging
Password Security	SHA-256 Hashing	Secure storage of student passwords

Table 4: Technology Stack

3 Key Terminology

3.1 Important Technical Terms

Term	Category	Explanation
Server	Networking	The teacher's computer that accepts connections from students and manages all classroom activities
Client	Networking	A student's computer that connects to the teacher's server to participate in classroom activities
Socket	Networking	A software endpoint that allows two programs to communicate over a network
Port (8888)	Networking	A virtual communication channel number that identifies where the server is listening
Thread	Concurrency	A separate execution path that allows the server to handle multiple students simultaneously without blocking
Heartbeat	Monitoring	A periodic signal (every 5 seconds) sent by students to indicate they are still connected
Session	Security	A temporary connection period representing a student's logged-in state
Session Token	Security	A unique identifier assigned to each logged-in student for authentication
Broadcast	Communication	Sending a single message to ALL connected students simultaneously
Quiz Queue	Quiz System	A temporary list holding multiple questions before they are sent to students
ToggleGroup	JavaFX UI	A JavaFX component that groups radio buttons so only one can be selected at a time
ProgressBar	JavaFX UI	A visual indicator showing the student's quiz completion percentage

TitledPane	JavaFX UI	A collapsible panel used to organize action buttons in the teacher dashboard
ListView	JavaFX UI	A scrollable list component used to display connected students
TextArea	JavaFX UI	A multi-line text area used for chat displays and logs
SHA-256	Security	A cryptographic hash function that converts passwords into fixed-length strings for secure storage
JDBC	Database	Java Database Connectivity - the API for connecting Java to databases

3.2 Message Protocol Terms

Message Type	**Direction**	**Purpose**
`LOGIN:user:pass`	Student → Teacher	Student attempts to authen
`LOGIN_SUCCESS:token:name:id`	Teacher → Student	Login successful, returns ses
`LOGIN_FAILED:msg`	Teacher → Student	Authentication failed
`HEARTBEAT`	Student → Teacher	Indicates student is still act
`HEARTBEAT_OK`	Teacher → Student	Acknowledgment of heartbe
`ANNOUNCEMENT:msg`	Teacher → Student	Teacher sends announcemer
`QUIZ:info|q|A:o1|B:o2|C:o3|D:o4|CORRECT:X`	Teacher → Student	Quiz question transmission
`QUIZ_ANSWER:X`	Student → Teacher	Student submits answer (A,
`LOCK_SCREEN`	Teacher → Student	Disable student's exam pan
`UNLOCK_SCREEN`	Teacher → Student	Enable student's exam pane
`QUIZ_COMPLETE`	Teacher → Student	Indicates quiz has ended
`REGISTER:...`	Student → Teacher	New student registration
`REGISTER_SUCCESS`	Teacher → Student	Registration successful
`ATTENDANCE:PRESENT`	Student → Teacher	Marks student attendance

Table 6: Network Message Types and Purposes

4 Features and Functionality

4.1 Teacher Dashboard Features

4.1.1 Real-time Student Monitoring

- Displays list of all connected students
- Shows **Active** status (student sending heartbeats)
- Shows **Inactive** status (no heartbeat for 10+ seconds)
- Updates automatically every 2 seconds
- Shows real-time student count

4.1.2 Attendance Tracking

- Automatically records when students log in
- Logs logout time when session ends
- Manual "Mark All Present" override option
- Generates detailed attendance report with:
  - List of students present/absent
  - Login timestamps
  - Attendance percentage

4.1.3 Quiz Management System

- Create multi-question quizzes
- Each question has 4 multiple-choice options
- Questions are added to a queue before sending
- Send all questions at once with automatic delay
- Students receive questions one by one
- Automatic grading of student responses
- View real-time answer collection

4.1.4 Announcement System

- Instant broadcast to all connected students
- Messages appear as formatted popup notifications
- Timestamp displayed with each announcement
- History stored in database

4.1.5 Classroom Control Features

- **Lock Screens:** Disables all student exam panels
- **Unlock Screens:** Re-enables exam panels
- **Reset Lock:** Emergency unlock for all students
- Prevents cheating during examinations

4.1.6 Report Generation

- Generate comprehensive classroom reports
- Includes: online count, registered students, activity log
- Can save reports as text files
- Includes quiz response summaries

4.2 Student Portal Features

4.2.1 Authentication System

- Secure login with username and password
- Self-registration for new students
- Password hashing using SHA-256
- Session management with unique tokens

4.2.2 Quiz Taking Interface

- Dedicated exam panel appears when quiz starts
- MCQ selection using radio buttons (A/B/C/D)
- **Submit Answer** button to save response
- **Previous/Next** buttons for navigation
- Progress bar showing completion percentage
- **Finish Quiz** button to see final results
- Real-time score display during quiz

4.2.3 Real-time Score Display

- Shows current score: `3/5 (60%)`
- Color-coded feedback:
  - Green: 70% and above (Good)
  - Orange: 50-69% (Average)
  - Red: Below 50% (Needs Improvement)

4.2.4 Announcement Reception

- Receives teacher announcements instantly
- Displays in formatted box with borders
- Timestamp shown for each announcement
- Appears in chat area

4.2.5 Screen Lock/Unlock

- Teacher can lock exam panel during test
- Student sees warning dialog when locked
- Exam panel becomes grayed out
- Cannot answer questions until unlocked
- Lock status indicator in header (/)

4.2.6 User Preferences

- **Immediate Feedback:** See correct/incorrect after each answer
- **Show Correct Answer:** Learn from mistakes immediately
- Preferences are saved between sessions

5 How the System Works

5.1 System Workflow Diagram

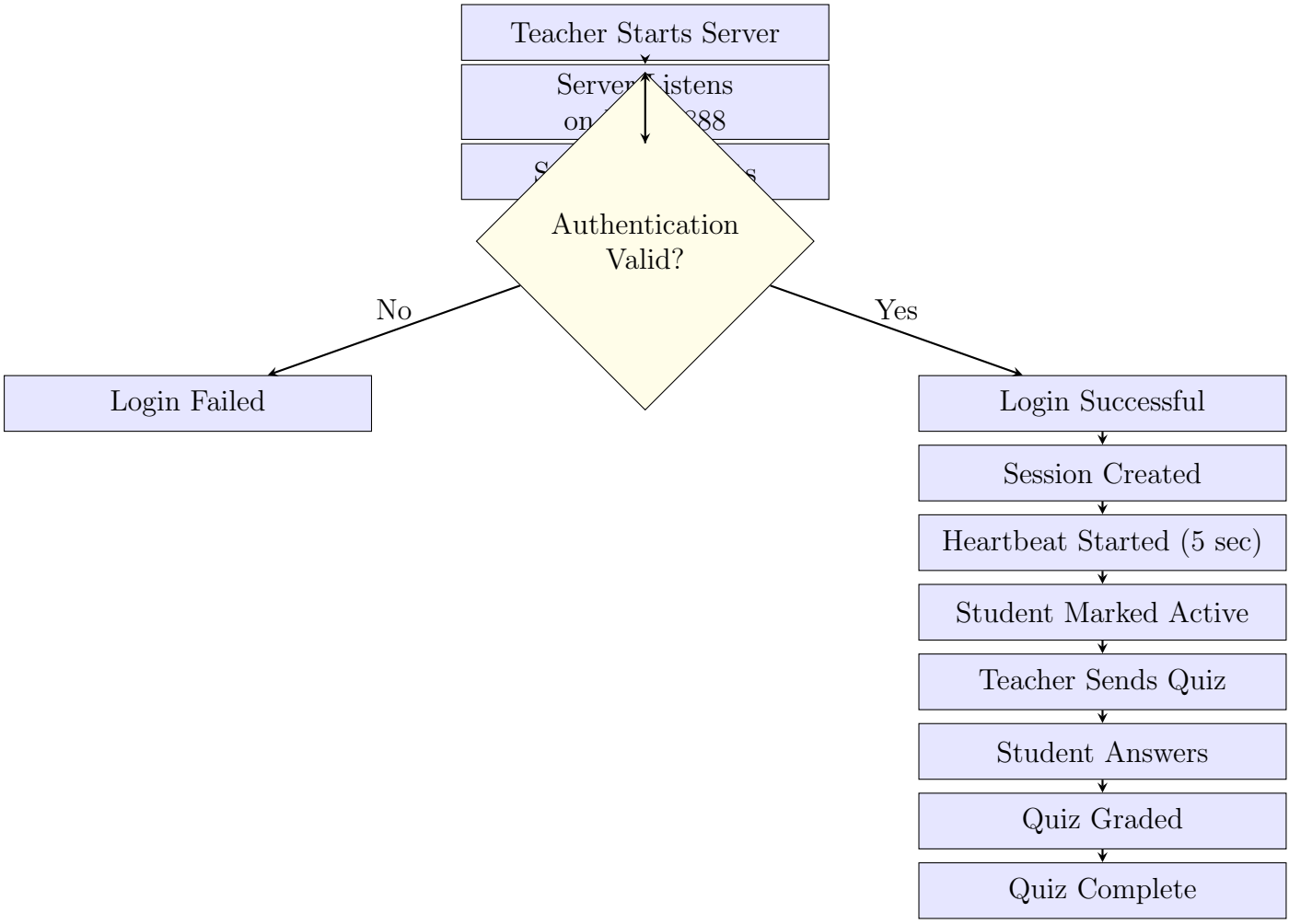


Figure 2: System Workflow Diagram

5.2 Step-by-Step Process Explanation

5.2.1 Step 1: Server Startup

1. Teacher launches `ServerMain.java`
2. Server creates a `ServerSocket` on port 8888
3. Server begins listening for incoming connections
4. PostgreSQL database connection is established
5. Teacher dashboard UI becomes visible

Code Explanation:

```
ServerSocket serverSocket = new ServerSocket(8888);
while (isRunning) {
    Socket clientSocket = serverSocket.accept();
    new StudentHandler(clientSocket).start();
}
```

5.2.2 Step 2: Student Connection

1. Student launches `StudentClientFX.java`
2. Client creates a `Socket` to teacher's IP on port 8888
3. Server accepts the connection
4. Server creates a new thread for this student
5. Authentication process begins

Code Explanation:

```
Socket socket = new Socket("localhost", 8888);
PrintWriter out = new PrintWriter(socket.getOutputStream(), true);
BufferedReader in = new BufferedReader(new InputStreamReader(socket.
    getInputStream()));
```

5.2.3 Step 3: Authentication

1. Server sends `AUTH_REQUIRED` to client
2. Client responds with `LOGIN:username:password`
3. Server validates credentials against database
4. On success: `LOGIN_SUCCESS:token:name:id`
5. On failure: `LOGIN_FAILED:Invalid credentials`

5.2.4 Step 4: Heartbeat Mechanism

1. After login, student starts a thread that sends `HEARTBEAT` every 5 seconds
2. Server receives heartbeat and updates `lastHeartbeat` timestamp
3. Server sends `HEARTBEAT_OK` acknowledgment
4. Teacher dashboard shows Active if heartbeat within 15 seconds
5. If no heartbeat for 10+ seconds, shows Inactive

Code Explanation:

```
// Student heartbeat thread
new Thread(() -> {
    while (connected) {
        Thread.sleep(5000);
        out.println("HEARTBEAT");
    }
}).start();

// Server checks activity
public boolean isActive() {
    return (System.currentTimeMillis() - lastHeartbeat) < 15000;
}
```

5.2.5 Step 5: Quiz Creation and Sending

1. Teacher clicks "Create Multi-Question Quiz"
2. Enters question text and 4 options
3. Selects correct answer (A/B/C/D)
4. Clicks "Add to Quiz" to add to queue
5. Repeats for additional questions
6. Clicks "Send All Questions" when ready
7. Server sends each question with 500ms delay

Quiz Message Format:

```
QUIZ:1/5|What is Java?|A:Programming language|B:Coffee|C:Island|D:None|
    CORRECT:A
```

5.2.6 Step 6: Student Quiz Taking

1. Student receives quiz message
2. Exam panel becomes visible
3. Student selects answer using radio buttons
4. Clicks "Submit Answer"
5. Answer sent to server: `QUIZ_ANSWER:A`
6. If immediate feedback ON: shows correct/incorrect
7. Student uses Next/Previous to navigate
8. Clicks "Finish Quiz" when complete
9. Server returns final score and grade

5.2.7 Step 7: Screen Lock During Exam

1. Teacher clicks "Lock Screens"
2. Server broadcasts `LOCK_SCREEN` to all students
3. Student exam panel becomes disabled and grayed out
4. All buttons and radio buttons disabled
5. Student sees lock warning
6. Teacher clicks "Unlock Screens" to re-enable

6 Database Design

6.1 Entity Relationship Diagram

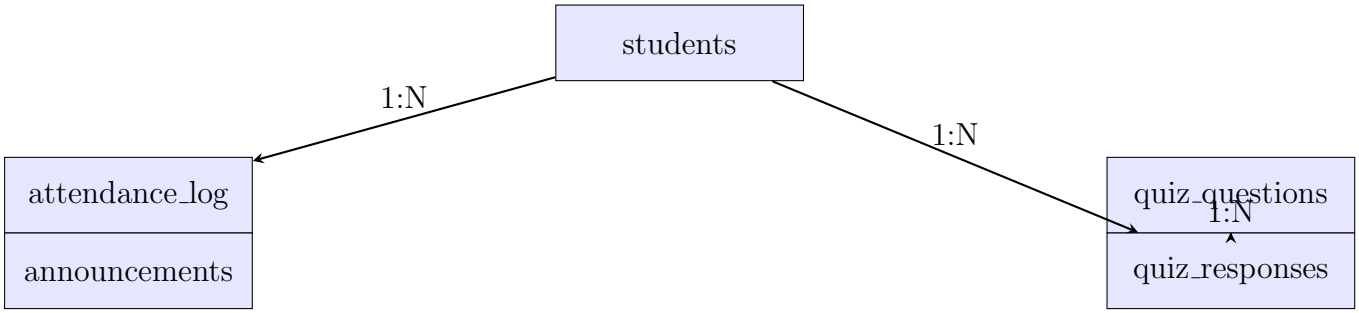


Figure 3: Entity Relationship Diagram

6.2 Table Structures

6.2.1 students Table

Stores all registered student information.

Column	Type	Constraint	Purpose
student_id	SERIAL	PRIMARY KEY	Unique identifier for each student
username	VARCHAR(50)	UNIQUE NOT NULL	Login username
password	VARCHAR(100)	NOT NULL	Hashed password (SHA-256)
full_name	VARCHAR(100)	NOT NULL	Student's full name
email	VARCHAR(100)	-	Contact email address
class_name	VARCHAR(50)	-	Class or section name
is_active	BOOLEAN	DEFAULT false	Currently online status
last_login	TIMESTAMP	-	Last login timestamp
registered_date	TIMESTAMP	DEFAULT NOW()	Account creation date
role	VARCHAR(20)	DEFAULT 'student'	User role (student/teacher)

Table 7: students Table Structure

6.2.2 attendance_log Table

Records all student login and logout events.

Column	Type	Constraint	Purpose
log_id	SERIAL	PRIMARY KEY	Unique log entry identifier
student_id	INT	FOREIGN KEY	References students(student_id)
action	VARCHAR(20)	NOT NULL	'LOGIN', 'LOGOUT', or 'MANUAL_PRESENT'
action_time	TIMESTAMP	DEFAULT NOW()	When the action occurred

Table 8: attendance_log Table Structure

6.2.3 quiz_questions Table

Stores all quiz questions created by the teacher.

Column	Type	Constraint	Purpose
question_id	SERIAL	PRIMARY KEY	Unique question identifier
question_text	TEXT	NOT NULL	The question text
option_a	VARCHAR(500)	NOT NULL	Option A text
option_b	VARCHAR(500)	NOT NULL	Option B text
option_c	VARCHAR(500)	-	Option C text (optional)
option_d	VARCHAR(500)	-	Option D text (optional)
correct_answer	CHAR(1)	NOT NULL	'A', 'B', 'C', or 'D'
created_time	TIMESTAMP	DEFAULT NOW()	When question was created

Table 9: quiz_questions Table Structure

6.2.4 quiz_responses Table

Stores student answers to quiz questions.

Column	Type	Constraint	Purpose
response_id	SERIAL	PRIMARY KEY	Unique response identifier
student_id	INT	FOREIGN KEY	References students(student_id)
question_id	INT	FOREIGN KEY	References quiz_questions(question_id)
student_answer	CHAR(1)	NOT NULL	Student's answer (A/B/C/D)
is_correct	BOOLEAN	NOT NULL	Whether the answer was correct
submitted_time	TIMESTAMP	DEFAULT NOW()	When answer was submitted

Table 10: quiz_responses Table Structure

6.2.5 announcements Table

Stores all announcements sent by the teacher.

Column	Type	Constraint	Purpose
announcement_id	SERIAL	PRIMARY KEY	Unique announcement identifier
message	TEXT	NOT NULL	Announcement text content
sent_time	TIMESTAMP	DEFAULT NOW()	When announcement was sent

Table 11: announcements Table Structure

7 Installation Guide

7.1 System Requirements

Component	Minimum Requirement
Operating System	Windows 10/11, Ubuntu 20.04+, macOS 11+
RAM	4 GB (8 GB recommended)
Storage	1 GB free space
Java Version	JDK 11 or higher
PostgreSQL	Version 12 or higher
Network	TCP/IP connectivity between teacher and students
Screen Resolution	1366 x 768 or higher

Table 12: System Requirements

7.2 Step-by-Step Installation

7.2.1 Step 1: Install Java (Ubuntu)

```
sudo apt update
sudo apt install openjdk-17-jdk
java -version
```

7.2.2 Step 2: Install PostgreSQL

```
sudo apt install postgresql postgresql-contrib
sudo systemctl start postgresql
sudo systemctl enable postgresql
```

7.2.3 Step 3: Create Database

```
sudo -u postgres psql
```

Then run the SQL commands in Appendix A.

7.2.4 Step 4: Download PostgreSQL JDBC Driver

```
cd ~/Downloads
wget https://jdbc.postgresql.org/download/postgresql-42.7.1.jar
```

7.2.5 Step 5: Install NetBeans IDE

```
# Download from https://netbeans.apache.org/
# Or install via snap
sudo snap install netbeans --classic
```

7.2.6 Step 6: Create Project in NetBeans

1. Open NetBeans IDE
2. File → New Project → Java with Ant → JavaFX Application
3. Project Name: `ClassroomServer`
4. Click Finish
5. Repeat for `StudentClientFX`

7.2.7 Step 7: Add JDBC Driver

1. Right-click on project → Properties
2. Click Libraries on left side
3. Click Add JAR/Folder
4. Select `postgresql-42.7.1.jar`
5. Click OK

7.2.8 Step 8: Add Source Code

1. Replace `ServerMain.java` with provided code
2. Replace `StudentClientFX.java` with provided code

7.2.9 Step 9: Update Database Password

In `DatabaseConnection` class, change:

```
1 private static final String PASSWORD = "your_postgres_password";
```

8 How to Run the Project

8.1 Running the Teacher Server

8.1.1 From NetBeans:

1. Open `ClassroomServer` project
2. Click Run → Run Project (or press F6)
3. Click "Start Server" button in the dashboard

8.1.2 From Terminal:

```
cd ~/NetBeansProjects/ClassroomServer
ant jar
java -cp "dist/ClassroomServer.jar:lib/postgresql-42.7.1.jar"
    classroomserver.ServerMain
```

8.2 Running the Student Client

8.2.1 From NetBeans:

1. Open `StudentClientFX` project
2. Click Run → Run Project (or press F6)

8.2.2 From Terminal:

```
cd ~/NetBeansProjects/StudentClientFX
ant jar
java -jar dist/StudentClientFX.jar
```

8.3 First Time Login

1. Click "Register here" on login screen
2. Fill in:
   - Username: `student1`
   - Password: `password123`
   - Full Name: `John Doe`
   - Email: `john@example.com`
   - Class: `Computer Science`
3. Click Register
4. Login with new credentials

Username	teststudent
Password	password

Table 13: Test Account Credentials

8.4 Test Account

8.5 Running Multiple Students

1. Open first terminal: run StudentClientFX
2. Open second terminal: run StudentClientFX again
3. Each instance connects as a different student
4. Teacher dashboard shows all connected students

9 Important Code Explanations

9.1 Multithreading - Handling Multiple Students

```
// Each student runs in a separate thread
public class StudentAuthHandler extends Thread {
    public void run() {
        // This code runs independently for each student
        while (connected) {
            String message = in.readLine();
            processMessage(message);
        }
    }
}

// Server accepts connections in a loop
while (isRunning) {
    Socket studentSocket = serverSocket.accept();
    new StudentAuthHandler(studentSocket, this).start();
}
```

Why this is important: Without threads, each student would have to wait for others to finish. Threads allow simultaneous handling.

9.2 Socket Communication

```
// Server side - listening
ServerSocket serverSocket = new ServerSocket(8888);
Socket client = serverSocket.accept();

// Reading and writing
BufferedReader in = new BufferedReader(
    new InputStreamReader(client.getInputStream())
);
PrintWriter out = new PrintWriter(
    client.getOutputStream(), true
);

// Send message
out.println("HELLO STUDENT");

// Receive message
String message = in.readLine();
```

9.3 Heartbeat Mechanism

```
// Student sends heartbeat every 5 seconds
new Thread(() -> {
    while (connected) {
        Thread.sleep(5000);
        out.println("HEARTBEAT");
    }
}).start();

```

```
// Server tracks last heartbeat
session.lastHeartbeat = System.currentTimeMillis();

// Check if student is active
public boolean isActive() {
    return (System.currentTimeMillis() - lastHeartbeat) < 15000;
}
```

9.4 Screen Lock Mechanism

```
// Teacher sends lock command
broadcastToAll("LOCK_SCREEN");

// Student receives and locks exam panel
if (message.equals("LOCK_SCREEN")) {
    screenLocked = true;

    // Disable all exam panel components
    examPanel.setDisable(true);
    examPanel.setStyle("-fx-opacity: 0.5;");

    for (RadioButton rb : optionButtons) {
        rb.setDisable(true);
    }

    prevBtn.setDisable(true);
    nextBtn.setDisable(true);
    submitAnswerBtn.setDisable(true);
    finishBtn.setDisable(true);
}

// Unlock command re-enables everything
if (message.equals("UNLOCK_SCREEN")) {
    screenLocked = false;
    examPanel.setDisable(false);
    examPanel.setStyle("-fx-opacity: 1;");
    // Re-enable all components
}
```

9.5 Password Hashing (SHA-256)

```
private String hashPassword(String password) {
    try {
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        byte[] hash = md.digest(password.getBytes("UTF-8"));
        StringBuilder hexString = new StringBuilder();

        for (byte b : hash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (Exception e) {
        return password;
```

```
    }
}
```

Example: Password "password" becomes: `5e884898da28047151d0e56f8dc6292773603d0d6aabbdd`

9.6 Quiz Message Parsing

```
// Example: "QUIZ:1/5|What is Java?|A:Language|B:Coffee|C:Island|D:None
   |CORRECT:A"
String[] parts = message.substring(5).split("\\|");
String qInfo = parts[0];           // "1/5"
String question = parts[1];        // "What is Java?"
String optA = parts[2].substring(2); // "Language"
String optB = parts[3].substring(2); // "Coffee"
String optC = parts[4].substring(2); // "Island"
String optD = parts[5].substring(2); // "None"
String correct = parts[6].substring(8); // "A"

String[] qParts = qInfo.split("/");
int qNum = Integer.parseInt(qParts[0]);   // 1
int total = Integer.parseInt(qParts[1]);  // 5
```

9.7 JavaFX UI Components Used

```
// ListView - displays list of students
ListView<String> studentListView = new ListView<>();

// ProgressBar - shows quiz completion
ProgressBar progressBar = new ProgressBar(0);
progressBar.setProgress(0.6);  // 60% complete

// ToggleGroup - groups radio buttons
ToggleGroup answerGroup = new ToggleGroup();
RadioButton optionA = new RadioButton();
optionA.setToggleGroup(answerGroup);

// TitledPane - collapsible section
TitledPane pane = new TitledPane("Title", contentVBox);
pane.setExpanded(true);

// TextArea - multi-line text area
TextArea logArea = new TextArea();
logArea.setEditable(false);
```

10 Troubleshooting

10.1 Common Errors and Solutions

Error Message	Cause	Solution
`Address already in use`	Port 8888 is occupied	Change PORT to 8889 in ServerMain.java
`No suitable driver found`	PostgreSQL JDBC not added	Add postgresql-42.7.1.jar to Libraries
`Connection refused`	PostgreSQL not running	`sudo systemctl start postgresql`
`Authentication failed`	Wrong database password	Check password in DatabaseConnection class
`Table doesn't exist`	Database not created	Run CREATE TABLE commands in Appendix A
`ClassNotFoundException`	Missing JDBC driver	Download and add postgresql-42.7.1.jar
`SocketException: Connection reset`	Student disconnected abruptly	Normal - student closed application
`JavaFX runtime components missing`	JavaFX not configured	Ensure JavaFX is added to project
Students not receiving messages	Firewall blocking port	`sudo ufw allow 8888`
Exam panel not showing	Quiz not sent correctly	Check server console for errors
Next/Previous buttons not working	Questions not fully loaded	Wait for all questions to arrive
Screen lock not working	Network delay	Wait 1 second, try again

10.2 Debugging Tips

1. **Check server console:** Look for error messages in NetBeans output
2. **Verify PostgreSQL:** `sudo systemctl status postgresql`
3. **Test database connection:** Run a simple SELECT query
4. **Check port availability:** `netstat -tulpn | grep 8888`
5. **Enable debug printing:** Add `System.out.println()` in message handlers
6. **Check firewall:** `sudo ufw status`

10.3 Port Configuration

If port 8888 is occupied, change it in both files:

```java
// In ServerMain.java
private static final int PORT = 8889;

// In StudentClientFX.java (connection line)
socket = new Socket("localhost", 8889);
```

10.4 Firewall Configuration

```
# Allow port 8888 through Ubuntu firewall
sudo ufw allow 8888

# Check firewall status
sudo ufw status

# If using iptables
sudo iptables -A INPUT -p tcp --dport 8888 -j ACCEPT
```

11 Conclusion

11.1 Project Summary

The Real-time Classroom Monitoring Platform successfully demonstrates the integration of multiple advanced Java programming concepts into a practical, functional application.

11.2 What Has Been Achieved

Feature	Status
Real-time student monitoring	Complete
Automatic attendance tracking	Complete
Multi-question quiz system	Complete
Announcement broadcasting	Complete
Screen lock/unlock functionality	Complete
Student registration/login	Complete
Report generation	Complete
Multi-threaded server	Complete
PostgreSQL database integration	Complete
JavaFX GUI with CSS styling	Complete

Table 15: Feature Completion Status

11.3 Skills Demonstrated

- **Java Programming:** Advanced object-oriented programming concepts
- **JavaFX:** Modern GUI development with CSS styling
- **Socket Programming:** TCP/IP network communication
- **Multithreading:** Concurrent processing of multiple clients
- **Database Design:** PostgreSQL schema design and relationships
- **JDBC:** Database connectivity and CRUD operations
- **UI/UX Design:** User-friendly interface design
- **Security:** Password hashing and session management

11.4 Difficulties Encountered and Solutions

11.5 Future Enhancements

Potential improvements for future versions:

1. Video conferencing integration
2. Screen sharing capabilities

Difficulty	Solution
Handling multiple student connections	Implemented multithreading with separate thread per student
Real-time status updates	Implemented heartbeat mechanism (every 5 seconds)
Quiz transmission order	Added 500ms delay between sending questions
Screen lock not affecting exam panel	Disabled all exam panel components when locked
Database connection errors	Added comprehensive error handling and logging
Next/Previous buttons navigation	Maintained question index and reloaded display

Table 16: Difficulties and Solutions

3. Mobile application (Android/iOS)
4. Advanced analytics dashboard
5. AI-powered question generation
6. Voice commands for teachers
7. Cloud-based deployment
8. Export reports as PDF/Excel
9. Automated email notifications
10. Parent portal for viewing progress

A Appendix A: Complete Database Schema

```sql
-- Drop existing tables (if needed)
DROP TABLE IF EXISTS quiz_responses CASCADE;
DROP TABLE IF EXISTS quiz_questions CASCADE;
DROP TABLE IF EXISTS attendance_log CASCADE;
DROP TABLE IF EXISTS announcements CASCADE;
DROP TABLE IF EXISTS students CASCADE;

-- Create students table
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    class_name VARCHAR(50),
    is_active BOOLEAN DEFAULT false,
    last_login TIMESTAMP,
    registered_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    role VARCHAR(20) DEFAULT 'student'
);

-- Create attendance_log table
CREATE TABLE attendance_log (
    log_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,
    action VARCHAR(20) NOT NULL,
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create announcements table
CREATE TABLE announcements (
    announcement_id SERIAL PRIMARY KEY,
    message TEXT NOT NULL,
    sent_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create quiz_questions table
CREATE TABLE quiz_questions (
    question_id SERIAL PRIMARY KEY,
    question_text TEXT NOT NULL,
    option_a VARCHAR(500) NOT NULL,
    option_b VARCHAR(500) NOT NULL,
    option_c VARCHAR(500),
    option_d VARCHAR(500),
    correct_answer CHAR(1) NOT NULL,
    created_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create quiz_responses table
CREATE TABLE quiz_responses (
    response_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id) ON DELETE CASCADE,
    question_id INT REFERENCES quiz_questions(question_id) ON DELETE
   CASCADE,
    student_answer CHAR(1) NOT NULL,
```

```
    is_correct BOOLEAN NOT NULL,
    submitted_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert default teacher account
INSERT INTO students (username, password, full_name, role)
VALUES ('teacher', 'teacher123', 'System Teacher', 'teacher');

-- Insert sample test student
INSERT INTO students (username, password, full_name, class_name, role)
VALUES ('teststudent', '5
   e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8',
        'Test Student', 'Class A', 'student');

-- Verify tables
SELECT * FROM students;
```

B Appendix B: Sample Output

B.1 Teacher Dashboard Console Output

```
[14:30:15]  Server started on port 8888
[14:30:15]  Waiting for student logins...
[14:30:20]  New connection from: 127.0.0.1
[14:30:20]  John Doe logged in
[14:30:25]  New connection from: 127.0.0.1
[14:30:25]  Jane Smith logged in
[14:31:00]  Question added. Total: 1
[14:31:05]  Question added. Total: 2
[14:31:10]  Question added. Total: 3
[14:31:15]  SENT: Question 1/3
[14:31:16]  SENT: Question 2/3
[14:31:17]  SENT: Question 3/3
[14:31:18]  All 3 questions sent to students!
[14:32:00]  Report generated
```

B.2 Student Portal Output

```
[14:30:22]   Connected to classroom!
[14:30:22]   Welcome John Doe!
[14:31:15]   Quiz started! Total questions: 3
[14:31:20]   Question 1: Correct!
[14:31:25]   Answer saved for Question 1
[14:31:30]   Moved to Question 2
[14:31:35]   Question 2: Incorrect.
[14:31:40]   Answer saved for Question 2
[14:31:45]   Moved to Question 3
[14:31:50]   Question 3: Correct!
[14:31:55]   Quiz Complete!
[14:31:55]  Score: 2/3 (67%) - C (Satisfactory)
```

C Appendix C: Project File Structure

```
ClassroomServer/
 src/
    classroomserver/
        ServerMain.java          (Teacher Dashboard - main class)
        StudentAuthHandler.java   (Handles student connections)
        DatabaseConnection.java   (PostgreSQL connection)
 lib/
    postgresql-42.7.1.jar        (JDBC driver)
 build/
    classes/                      (Compiled class files)
 dist/
    ClassroomServer.jar           (Executable JAR)
 build.xml                         (Ant build file)

StudentClientFX/
 src/
    studentclientfx/
        StudentClientFX.java      (Student Portal - main class)
 build/
    classes/                      (Compiled class files)
 dist/
    StudentClientFX.jar           (Executable JAR)
 build.xml                         (Ant build file)

Database/
 classroom_monitor/                (PostgreSQL database)
    students                      (Table)
    attendance_log                (Table)
    quiz_questions                (Table)
    quiz_responses                (Table)
    announcements                 (Table)
```

References

1. Oracle Corporation. (2024). *Java Documentation*. https://docs.oracle.com/en/java/
2. OpenJDK. (2024). *JavaFX Documentation*. https://openjfx.io/
3. PostgreSQL Global Development Group. (2024). *PostgreSQL Documentation*. https://www.postgresql.org/docs/
4. Apache NetBeans. (2024). *NetBeans IDE Documentation*. https://netbeans.apache.org/
5. Horstmann, C. S. (2019). *Core Java Volume I - Fundamentals*. Pearson Education.
6. Harold, E. R. (2013). *Java Network Programming*. O'Reilly Media.

— END OF DOCUMENTATION —